

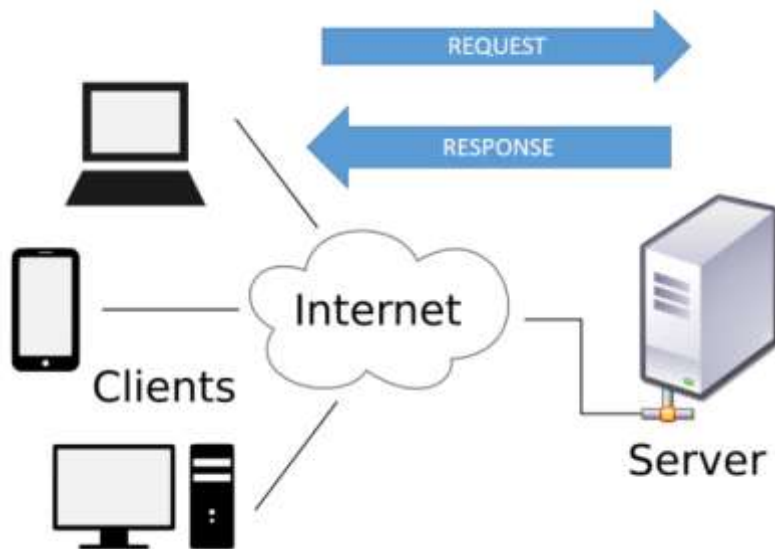
Android Socket

Contents

- **Socket concept**
- **Socket implementation**

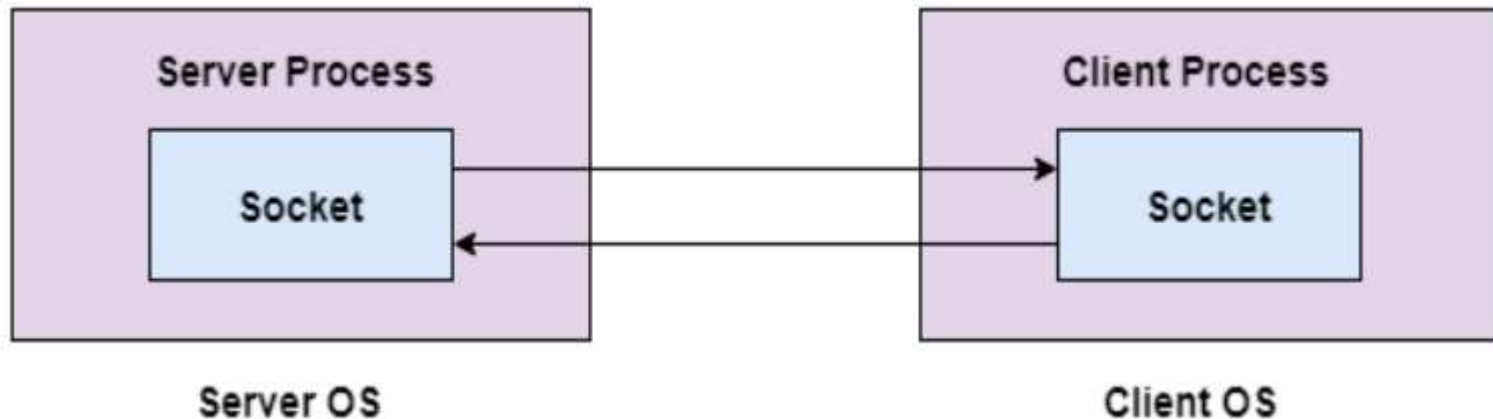
Client/Server communication

- Client/Server communication involves two components, namely a client and a server.
- The clients send requests to the server and the server responds to the client requests.

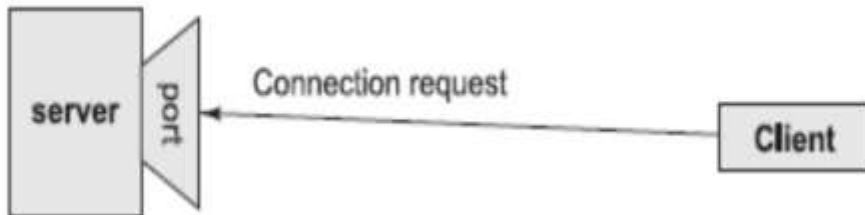


Socket

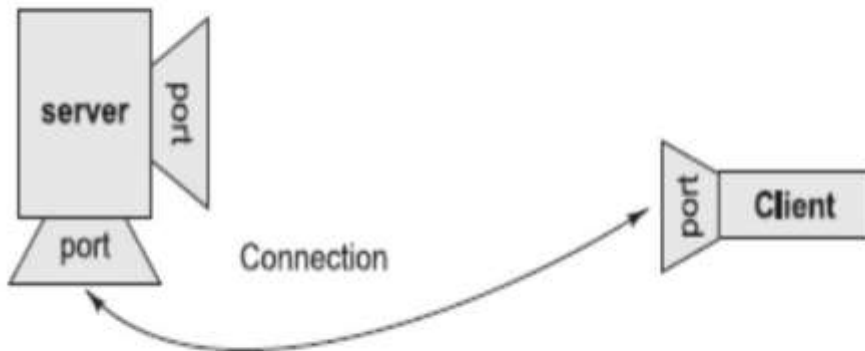
- A socket is a software endpoint that can plug into or be plugged into to create a bi-directional communication link between software processes
- A socket is a common interface for performing network communication
- Android's HTTP client library is using sockets to send and receive data



Socket connection

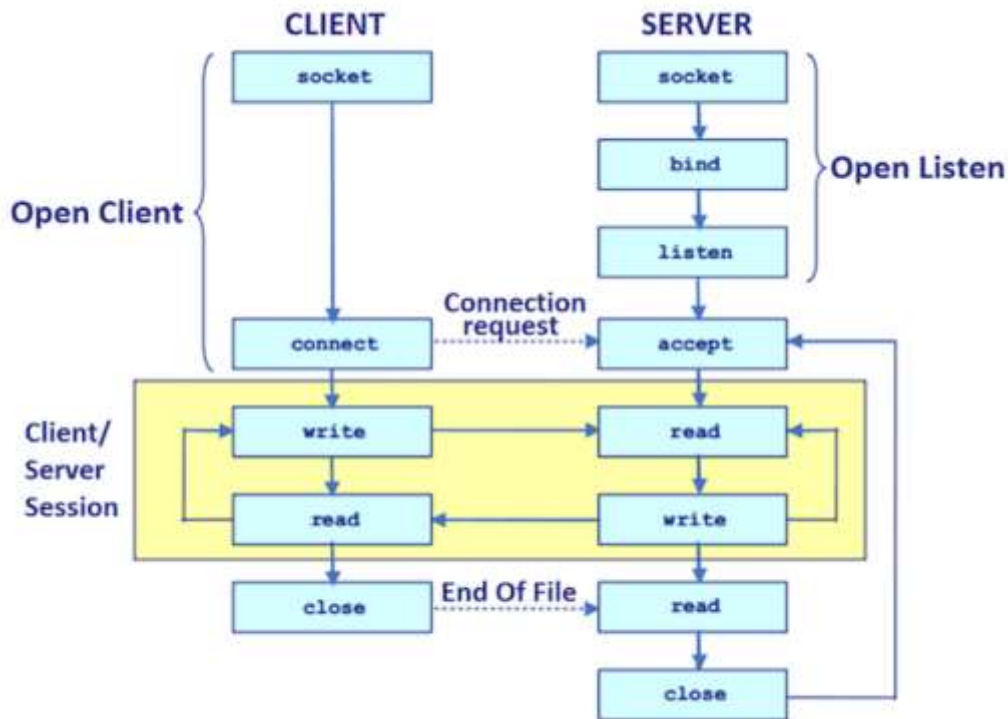


[a]: a client making a connection request to the server



[b]: session established with temporary ports used for two way communication.

Socket programming



SOCKET API

Socket programming methods

- **socket():** Create a socket
- **bind():** bind a socket to a local IP address and port #
- **listen():** passively waiting for connections
- **connect():** initiating connection to another socket
- **accept():** accept a new connection
- **write():** write data to a socket
- **read():** read data from a socket
- **sendto():** send a datagram to another UDP socket
- **recvfrom():** read a datagram from a UDP socket
- **close():** close a socket (tear down the connection)

InputStreams in Java

- An InputStream is a stream of incoming byte data
- An InputStream can be obtained from a Socket by using the `getInputStream()` method
- In order to read from a stream, you must create a byte buffer to read in data
- Each call to read on an InputStream fills your buffer with data and returns the number of bytes read

```
InputStream in = somesocket.getInputStream();
byte[] buffer = new byte[1024];
int bytesread = 0;

while( (bytesread = in.read(buffer,0,buffer.length)) != -1){

    //the buffer has been filled, do something with the data

}
```


OutputStreams in Java

- An OutputStream is a stream of outgoing byte data
- An OutputStream can be obtained from a Socket by using the `getOutputStream()` method
- You can write data to a stream by passing in a byte buffer of data
- You should use the `flush()` method if you want to make sure that the data you have written has been output to disk or sent to the other end of the socket

Steps to create Server program

- Open the server socket
`ServerSocket server = new ServerSocket(PORT)`
- Wait for the client request
`Socket client = server.accept()`
- Create I/O stream for communicating to the client
`output = new PrintWriter(client.getOutputStream());`
`input = new BufferedReader(new`
`InputStreamReader(client.getInputStream()));`
- Perform communication with client
Receive from client: `String message = input.readLine();`
Send to client: `output.write(message);`
- Close socket: `client.close`

Steps to create Client program

- Create a Socket object
Socket socket = new Socket(SERVER_IP, SERVER_PORT);
- Create I/O stream for communicating to server
output = new PrintWriter(socket.getOutputStream());
input = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
- Perform communication with server
Receive from client: String message = input.readLine();
Send to client: output.write(message);
- Close the socket
client.close

Usage

- **2 Android applications connect together by Socket:**
 - 1 Android application is Server
 - 1 Android application is Client



Classwork

- **Make a chat program by socket using socket.io library**

<https://socket.io/blog/native-socket-io-and-android/>

References

- <https://guides.codepath.com/android/Sending-and-Receiving-Data-with-Sockets>
- <https://www.tutorialspoint.com/sending-and-receiving-data-with-sockets-in-android>
- <https://github.com/nkzawa/socket.io-android-chat>
- <http://android-er.blogspot.com/2014/08/bi-directional-communication-between.html>